



RESEARCH EXECUTION PLAN

Scale-Dependent Benefits of Joint Pruning and Quantization

A controlled study across Pythia model scales with optional Qwen2.5 external validation

Working paper title

When Does Joint Compression Pay Off? A Controlled Scale Study of Sequential and Joint Pruning-Quantization in Decoder-Only Language Models

Study element	Finalised plan
Core models	Pythia-160M, Pythia-410M, Pythia-1B
Conditional extension	Pythia-1.4B if the pilot fits memory and time limits
External validation	Qwen2.5-0.5B base model, run only after the core sweep
Primary comparison	Sequential compression versus joint co-optimisation
Primary outcome	How joint gain changes with model scale
Execution window	Two to three weeks

NAVIGATION

Contents and Decision Summary

Section	Purpose
1	Study definition and research questions
2	Scope, model suite and controlled variables
3	Compression methodology and fairness protocol
4	Data, evaluation and CPU benchmarking
5	Experimental matrix and staged execution
6	Analysis plan and claims that are allowed
7	Repository, logging and reproducibility
8	Two-to-three-week schedule
9	Complete paper headings and subheadings
10	Figures, tables, risks and final checklists

The study in one paragraph

This project investigates whether the advantage of joint pruning and quantization changes as decoder-only language models become larger. The main experiment uses Pythia-160M, Pythia-410M and Pythia-1B because the Pythia suite was designed for controlled scaling research: models are provided across multiple sizes under a closely matched training setup, including the same data order within the suite [1,2]. A fourth Pythia-1.4B point is conditional on a hardware pilot. Qwen2.5-0.5B is an optional external validation model and is analysed through within-model relative retention rather than raw cross-tokenizer perplexity [3].

Core methodological choice

Use a layerwise post-training reconstruction approach for the full scale sweep. It is more realistic on a laptop than full-model quantization-aware training at 1B-1.4B parameters, while still allowing a genuine sequential-versus-joint comparison under the same calibration data, budgets and optimisation steps.

Mandatory versus optional work

Tier	Included work	Status
Mandatory core	160M, 410M and 1B; dense, pruning-only, quantization-only, sequential and joint; two final compression budgets; perplexity, downstream quality and CPU efficiency.	Must finish
Conditional extension	Pythia-1.4B after a memory/time pilot.	Add only if feasible
External validation	Qwen2.5-0.5B at one or two representative budgets.	Run last
Optional ablations	Reverse sequential order, extra bit-width, extra sparsity level, extra downstream tasks.	Only after core results

1. Study Definition

1.1 Background and motivation

Pruning and quantization reduce different forms of model cost. Pruning removes or suppresses weights, while quantization represents surviving weights with fewer bits. They are often applied as separate stages, but the first stage changes the weight distribution and sensitivity landscape seen by the second. Joint methods attempt to coordinate both decisions so that the pruning mask and quantized representation adapt to one another. Recent joint-compression work argues that co-optimisation can improve the compression-quality trade-off, but the engineering cost is higher and the size regime in which that added complexity becomes worthwhile remains underexplored [4,5].

1.2 Problem statement

Existing comparisons commonly ask whether a joint method beats a sequential method on one model or one architecture. That does not reveal whether the size of the model changes the magnitude of the joint advantage. A controlled scale sweep is therefore needed to determine whether joint gain is negligible at smaller scales, increases gradually, appears only under aggressive compression, or shows no consistent relationship with scale.

1.3 Research gap

The gap addressed by this paper is not the invention of a new universal compression algorithm. The contribution is a controlled empirical analysis of scale as the independent variable while the model family, evaluation protocol, compression budgets and implementation are held as constant as possible.

1.4 Aim

To determine how model scale influences the relative effectiveness of joint versus sequential pruning and quantization in decoder-only language models.

1.5 Primary research question

Primary RQ

How does model scale influence the quality and efficiency advantage of joint compression over sequential pruning and quantization in decoder-only Pythia language models?

1.6 Secondary research questions

- RQ2. Does the scale trend change between moderate and aggressive compression budgets?
- RQ3. Is any observed benefit driven by improved language-model perplexity, downstream task retention, or both?
- RQ4. Does a trend observed within Pythia also appear in Qwen2.5-0.5B when results are normalised to each model's dense baseline?
- RQ5. Do compressed checkpoints produce real CPU memory or latency benefits, or only theoretical parameter-count reductions?
- RQ6. How sensitive are conclusions to sequential order, calibration sample size and targeted layer set?

1.7 Hypotheses

ID	Hypothesis
H1	Joint gain increases with model scale because larger models may offer more redundant alternatives for coordinated pruning and quantization.
H2	Joint and sequential methods differ little under mild compression but separate more clearly under aggressive budgets.
H3	A joint method improves quality retention more reliably than it improves raw CPU latency, because sparse acceleration depends on backend support.
H4	The scale trend remains directionally similar on Qwen2.5-0.5B, although its absolute scores are not directly comparable to Pythia because architecture, training data and tokenizer differ.
H5	The reverse sequential order may outperform the default order at some budgets; therefore joint compression must be compared with the strongest feasible sequential baseline, not only one convenient order.

Do not assume a threshold

The paper may discuss an apparent transition only if the observed data support it. With three or four scale points, the defensible claim is a trend or change in effect size, not a universal parameter-count threshold.

1.8 Expected contributions

- A controlled scale sweep within one decoder-only model family.
- A compute-matched sequential-versus-joint comparison at equal sparsity and bit-width budgets.
- A scale-normalised definition of joint gain suitable for both accuracy-like metrics and perplexity.
- A distinction between theoretical compression and measured CPU deployment benefit.
- An optional cross-family validation on a modern Qwen base model.
- A fully reproducible experiment pipeline with raw logs, seeds, configs and benchmark scripts.

1.9 Scope and exclusions

In scope	Out of scope
Decoder-only base language models	Instruction tuning, RLHF or chat-alignment effects
Weight pruning and weight quantization	Knowledge distillation as a third compression axis
Post-training or short recovery optimisation	Training models from scratch
Pythia-controlled scale sweep	Claiming that all model families share one threshold
CPU inference benchmarking	Claiming sparse speedups when the runtime does not exploit sparsity
One external Qwen check	A full multi-family scaling law study

2. Scope, Model Suite and Controlled Variables

2.1 Core scale sweep

Model	Role	Priority	Reason
Pythia-160M	Low-scale anchor	Required	First full pipeline and debugging model
Pythia-410M	Middle scale	Required	Tests whether joint gain begins to change
Pythia-1B	Upper core scale	Required if pilot succeeds	Primary large point for the central claim
Pythia-1.4B	Additional upper point	Conditional	Improves trend resolution but is not allowed to delay the core paper
Qwen2.5-0.5B	External architecture	Optional, run last	Checks whether the direction is Pythia-specific

2.2 Why Pythia is the primary family

Pythia is suitable because it was released as a scaling suite with multiple model sizes and controlled training resources. The models span a wide parameter range, and the suite provides a consistent basis for comparing how behaviour changes with size [1,2]. This does not make every architectural quantity identical across sizes, but it removes far more confounds than comparing an encoder model with a decoder model or mixing unrelated model families.

2.3 Why Pythia-70M is excluded

The main sweep begins at 160M to preserve compute for the 1B point, which is more important for testing a possible increase in joint benefit. The 70M model is not described as scientifically useless; it is simply outside the minimum viable design. Add it only if early results suggest that the lower end of the curve is not adequately anchored.

2.4 Qwen external validation rules

- Use the base Qwen2.5-0.5B model, not the instruction-tuned variant, to remain closer to the base causal-LM setting [3].
- Run only after all mandatory Pythia experiments and plots are complete.
- Use the same compression implementation and the same two representative budgets where technically possible.
- Do not compare raw Qwen perplexity directly with raw Pythia perplexity because tokenizers and pretraining differ.
- Compare within-model quality retention, downstream score change and joint-versus-sequential effect size.

2.5 Independent, dependent and controlled variables

Type	Variable	Operational definition
Independent variable	Model scale	160M, 410M, 1B, optional 1.4B
Primary treatment	Compression strategy	Sequential versus joint
Compression budget	Sparsity and bit-width	Two frozen budgets selected in screening
Quality outcomes	Perplexity and downstream task scores	Dense-normalised and absolute values
Efficiency outcomes	Checkpoint size, peak RAM, prefill/decode latency, throughput	Measured on the same CPU runtime

Type	Variable	Operational definition
Controlled variables	Data, samples, sequence lengths, targeted layers, optimiser steps, random seeds, thread count	Identical unless explicitly ablated

2.6 Important scale confound: embeddings

Pythia model names reflect total model size, while the proportion of parameters in token embeddings and the language-model head changes with scale. If pruning and quantization target only transformer-block linear layers, the fraction of the total model compressed will differ by size. The analysis must therefore report both total parameters and targeted non-embedding parameters. The main scale x-axis should be the targeted non-embedding parameter count, with total model size shown as a secondary label.

Layer targeting rule

Primary experiments should prune and quantize transformer-block linear weights. Do not prune LayerNorm, biases or token embeddings. Quantizing embeddings and the language-model head may be included as a separate deployment ablation, but it must not silently differ across model sizes.

2.7 Decisions to freeze before full experiments

Decision	Required record
Model revisions	Pin exact Hugging Face commit/revision for every checkpoint.
Pythia variant	Use one consistent data variant across all sizes; never mix standard and deduplicated variants.
Target layers	List exact module patterns and exclusions.
Quantization	Freeze bit-width, symmetry, group size and scale granularity.
Pruning	Freeze global/layerwise rule, saliency score and mask update schedule.
Calibration data	Freeze dataset, sample indices, token count and sequence length.
Benchmark runtime	Freeze PyTorch/ONNX/other backend before reporting latency.
Seeds	Freeze exploratory and confirmatory seed policy.

3. Compression Methodology and Fairness Protocol

3.1 Method family selected for this study

The recommended implementation is layerwise post-training reconstruction. For each linear layer, calibration activations X are collected and the compressed weight representation is chosen to minimise the difference between the dense layer output XW and the compressed output. This avoids full end-to-end fine-tuning of 1B-1.4B models and allows the same method to be applied across the scale sweep on limited hardware.

Core layerwise objective

Dense layer output: $Y = XW$
 Compressed output: $Y_{\text{hat}} = X(M * Q_b(W))$
 Reconstruction loss: $L_{\text{rec}} = ||Y - Y_{\text{hat}}||_F^2$
 Constraints: $\text{sparsity}(M) = s, \text{bit_width}(Q_b) = b$
 Elementwise product: $*$

3.2 Dense baseline

- › Load the pinned pretrained checkpoint in a fixed precision.
- › Evaluate dense perplexity and downstream tasks before any compression.
- › Run the CPU benchmark protocol and save the raw baseline logs.
- › Record parameter counts, targeted parameter counts and checkpoint size.
- › Verify deterministic model loading and evaluation across two repeated runs.

3.3 Pruning-only baseline

Apply the selected pruning rule to the targeted transformer-block linear weights while leaving their numeric precision unchanged. The minimum viable baseline is activation-weighted magnitude pruning; a second-order method may be added only if it is already stable. The pruning-only result isolates damage caused by sparsity.

3.4 Quantization-only baseline

Quantize the same targeted weights without pruning. The preferred baseline is symmetric weight-only quantization using the same scale granularity and group size as the combined methods. Quantization-only results isolate damage caused by reduced precision.

3.5 Sequential pipeline

Primary sequential order: Prune -> Quantize

1. Collect calibration activations.
2. Compute pruning scores on the dense weights.
3. Create the target pruning mask.
4. Reconstruct or recover the surviving dense-valued weights.
5. Quantize the surviving weights using the fixed bit-width scheme.
6. Optionally perform the same number of local reconstruction steps allowed to the joint method.
7. Freeze and evaluate the compressed model.

3.6 Reverse sequential order

At one representative budget, run Quantize -> Prune as an ablation. This prevents the paper from comparing joint compression only against a weak ordering. If both sequential orders are feasible, define the best sequential result per model and budget as the stronger of the two.

3.7 Joint pipeline

Joint compression must allow the pruning decision and quantized representation to influence one another. A practical implementation is alternating layerwise optimisation: initialise a mask and quantization parameters, evaluate reconstruction error under both constraints, update mask scores using quantized or fake-quantized weights, re-estimate quantization scales on the surviving weights, and locally optimise surviving weights/scales for a fixed number of steps.

Joint alternating co-optimisation

Initialise mask M and quantizer Q_b.
 repeat K joint iterations:
 fake-quantize or quantize surviving weights
 compute reconstruction loss under current M and Q_b
 update mask/saliency while respecting target sparsity
 update quantization scales or codebook
 optimise surviving weights/scales for fixed local steps
 freeze M and Q_b, then evaluate

3.8 What qualifies as joint

Qualifies as joint	Does not qualify
Mask decisions are evaluated under quantized or fake-quantized weights.	Prune completely, freeze the result, then call ordinary PTQ.
Quantization scales/codebooks are re-estimated after mask changes.	Quantize completely, then prune without re-optimising quantization.
A common reconstruction objective is used while both constraints are active.	Run two unrelated scripts and label the final file joint.
The final mask and quantized weights are jointly refined for fixed steps.	Compare methods with different calibration data or optimisation budgets.

3.9 Recommended quantization configurations

Configuration	Definition	Role
Screening W8	8-bit weight-only, symmetric, per-channel or fixed groupwise	Stable moderate baseline
Screening W4	4-bit weight-only, symmetric, fixed group size across models	Aggressive precision baseline
Activation quantization	Not part of the core design	Adds a second major variable and backend dependence
Embedding/head quantization	Optional deployment ablation	Report separately from the main block-weight study

3.10 Recommended pruning configurations

Item	Default	Rationale
Screening sparsities	30%, 50%, 70% of targeted weights	Find mild, moderate and aggressive regimes
Primary granularity	Global unstructured or fixed blockwise mask	Keep identical across all Pythia sizes
Layer exclusions	Embeddings, LM head, norms and biases	Avoid destabilising components and preserve comparability
Structured pruning	Optional future work	It changes shapes and requires a different implementation/runtime

3.11 Fairness rules

- [] Equal target sparsity and bit-width for sequential and joint methods.
- [] Identical calibration examples, ordering, token count and sequence length.
- [] Equal total local optimisation steps and approximately equal objective evaluations.

- [] Identical targeted modules and layer processing order.
- [] Identical evaluation code, tokenizer, prompts and generation settings.
- [] The same seed set for direct comparisons.
- [] Hyperparameters tuned on the screening model only; do not tune separately for every scale unless that is a declared ablation.
- [] Any failed or diverged run remains in the experiment log with the failure reason.

Critical fairness point

If the joint method receives more recovery steps, more calibration tokens or model-specific tuning than the sequential baseline, a higher score cannot be attributed cleanly to joint optimisation.

3.12 Deployment representation versus scientific comparison

Unstructured sparsity may reduce the number of non-zero weights without producing faster CPU inference in a dense runtime. Report three separate quantities: theoretical non-zero parameter reduction, actual serialised checkpoint size, and measured CPU latency. Never infer speedup from sparsity alone. Prior pruning work has shown that speedups require suitable sparse kernels and deployment integration [6].

4. Data, Evaluation and CPU Benchmarking

4.1 Calibration and local reconstruction data

Use one fixed text corpus and one fixed set of tokenised sequences for all Pythia models. A practical plan is a held-out, reproducibly sampled language-modelling corpus with enough diversity to generate representative activations. Store the exact sample indices and tokenised tensors. The same calibration set must be used for sequential and joint methods at every scale.

Stage	Recommended size	Purpose
Smoke test	128-256 sequences	Validate activation capture and layer reconstruction
Screening	512-1,024 sequences	Choose two final compression budgets
Full compression	1,024-2,048 sequences, if feasible	Stable layerwise statistics across the scale sweep
Sensitivity ablation	25%, 50% and 100% of the chosen set	Measure calibration-size sensitivity on one model

4.2 Primary language-modelling evaluation

Use a standard held-out corpus such as WikiText-2 for perplexity. Perplexity is the primary model-quality metric because all core models are base causal language models. Use identical sequence construction and stride for all Pythia models. For Qwen, report perplexity only relative to its own dense baseline because tokenization differs.

4.3 Secondary downstream evaluation

Use a small, stable task set from the Language Model Evaluation Harness, which supports zero-shot and few-shot evaluation for autoregressive models [7]. Keep the task set small enough to finish on CPU or GPU without compromising the compression sweep.

Task	Capability	Metric	Priority
HellaSwag	Commonsense continuation	Accuracy / normalised accuracy	Required
PIQA	Physical commonsense	Accuracy / normalised accuracy	Required
ARC-Easy	Basic science reasoning	Accuracy / normalised accuracy	Required
LAMBADA	Long-range word prediction	Accuracy or perplexity	Optional
BoolQ	Reading comprehension	Accuracy	Optional

4.4 Quality metrics

- Absolute perplexity and change from dense baseline.
- Downstream task score and percentage-point change from dense baseline.
- Dense-normalised quality retention.
- Joint gain over the strongest sequential baseline.
- Failure rate or divergence count under aggressive compression.
- Optional token-level KL divergence or logits reconstruction error on a held-out calibration subset.

Normalised metrics

Higher-is-better retention: $R_{\text{high}} = \text{score_compressed} / \text{score_dense}$

Lower-is-better retention: $R_{\text{ppl}} = \text{PPL_dense} / \text{PPL_compressed}$

Accuracy joint gain: $JG_{\text{acc}} = \text{score_joint} - \text{score_best_sequential}$

Perplexity joint gain (%): $JG_{\text{ppl}} = 100 * (\text{PPL_best_seq} - \text{PPL_joint}) / \text{PPL_best_seq}$

4.5 Efficiency metrics

Metric	Operational definition
Checkpoint size	On-disk MB/GB of the actual deployable artefact
Effective bits per targeted weight	Includes scale/zero-point overhead where possible
Non-zero targeted parameters	Actual mask count, not nominal sparsity only
Peak resident RAM	Maximum process memory during inference
Prefill latency	Prompt processing time at fixed context lengths
Decode latency	Per-token or total time for fixed generated-token count
Throughput	Tokens per second, batch size 1 and optionally batch size 4
Compression time	Wall-clock time and peak GPU/CPU memory for each method

4.6 GPU use versus CPU evaluation

Correct interpretation

The study may use the laptop GPU for activation collection and compression. CPU-only evaluation means that final latency, throughput and memory measurements are made on the CPU under a fixed runtime. It does not require every optimisation step to run on the CPU.

4.7 CPU benchmarking protocol

- [] Plug the laptop into power and use a fixed performance mode.
- [] Record exact CPU model, core count, RAM, operating system and power profile.
- [] Close heavy background applications and allow the system to cool before each benchmark block.
- [] Pin the inference runtime, library versions and thread count.
- [] Use batch size 1 as the primary deployment setting.
- [] Benchmark at fixed prompt lengths, for example 128 and 512 tokens.
- [] Measure prefill and decode separately; generate a fixed number of new tokens, for example 64.
- [] Run warm-up iterations before timing.
- [] Collect at least 20-30 timed repetitions for lightweight models; use fewer only when a run is prohibitively slow and report the count.
- [] Report median, interquartile range and p95 latency.
- [] Randomise or rotate model order to reduce thermal and temporal bias.
- [] Do not compare models measured under different thread counts or backends.

4.8 Evaluation integrity checks

- [] Verify that the compressed checkpoint reloads independently from disk.
- [] Check that reported sparsity remains after serialisation and reload.
- [] Confirm that quantized weights use the intended bit-width rather than being silently dequantized to full precision.
- [] Log tokenizer revision and evaluation-task version.
- [] Manually inspect a small set of generations for catastrophic collapse.
- [] Use a held-out reconstruction subset to ensure the method is not overfitting the calibration samples.

5. Experimental Matrix and Staged Execution

5.1 Stage A: hardware and implementation pilot

Step	Action
A1	Load 160M, 410M and 1B; record CPU RAM and GPU VRAM.
A2	Capture activations for one representative layer at the target sequence length.
A3	Run one pruning-only, one quantization-only and one joint layer reconstruction.
A4	Measure projected full-model compression time.
A5	Decide whether 1.4B meets the go/no-go rule.
A6	Freeze software environment and checkpoint revisions.

5.2 Go/no-go rule for Pythia-1.4B

Add 1.4B only when all conditions are met

Peak VRAM remains below approximately 85% of available capacity; the projected full run fits the remaining schedule; the 1B core sweep is already stable; and adding 1.4B does not remove confirmatory seeds or analysis time.

5.3 Stage B: compression-budget screening

Run the screening grid on Pythia-160M first, then confirm promising budgets on Pythia-410M. The purpose is not to publish every combination; it is to select one moderate and one aggressive budget before the full scale sweep.

Candidate	Budget	Role
S1	30% sparsity + W8	Mild/moderate
S2	50% sparsity + W8	Moderate
S3	50% sparsity + W4	Aggressive precision
S4	70% sparsity + W4	Aggressive combined

Selection rule: choose two budgets that are technically stable, produce measurable but non-catastrophic quality loss, and show enough separation to test whether joint gain changes with scale. Freeze the choices before running 1B.

5.4 Stage C: full scale sweep

Condition	Configuration	Planned count
Dense	No pruning, reference precision	1 per model
Pruning only	Sparsity matching each selected combined budget	2 per model
Quantization only	Bit-width matching each selected combined budget	2 per model
Sequential P->Q	Selected budgets	2 per model
Joint	Selected budgets	2 per model
Reverse Q->P	One representative budget	1 per model or reduced ablation

5.5 Seed policy

Run type	Seeds	Purpose
Exploratory screening	1	Fast budget selection and debugging
Full matrix first pass	1	Complete coverage across models and conditions
Confirmatory central comparison	3 total seeds	Sequential versus joint at both budgets for 160M, 410M and 1B
Optional models/ablations	1 unless time permits	Secondary evidence only

5.6 Experiment naming convention

Run identifier

<family>_<size>_<method>_<sparsity>_<bits>_<seed>

Example: pythia_410m_joint_s50_w4_seed2

Example: pythia_1b_seq_pq_s50_w8_seed1

Example: qwen25_05b_joint_s50_w4_seed1

5.7 Minimum viable result set

- [] Dense baseline for all three mandatory Pythia sizes.
- [] Two frozen compression budgets.
- [] Pruning-only and quantization-only controls.
- [] Sequential P->Q and joint for every model/budget.
- [] Three-seed confirmation of the central sequential-versus-joint comparisons.
- [] WikiText-style perplexity, three downstream tasks and CPU benchmark metrics.
- [] A scale-effect plot with uncertainty.
- [] A limitations section that avoids threshold overclaiming.

5.8 Optional expansion order

- › First: reverse sequential order at the central budget.
- › Second: Pythia-1.4B.
- › Third: Qwen2.5-0.5B external validation.
- › Fourth: extra calibration-size or group-size ablation.
- › Last: additional downstream tasks.

6. Analysis Plan and Permissible Claims

6.1 Primary effect size

The central outcome is joint gain over the strongest sequential baseline at each scale and budget. Plot joint gain against the logarithm of targeted non-embedding parameter count. Show each seed and the mean/median with an uncertainty interval.

6.2 Scale analysis

- Describe the direction and magnitude of joint gain from 160M to 410M to 1B.
- Compare the moderate and aggressive budgets separately.
- Use a simple trend line only as a descriptive aid; three points do not establish a universal scaling law.
- If 1.4B is added, report whether it strengthens, flattens or reverses the trend.
- Do not treat Qwen as another point on the Pythia scale curve.

6.3 Statistical treatment

- Report mean and standard deviation, or median and range, across confirmatory seeds.
- Use paired comparisons when sequential and joint runs share the same seed and calibration sample.
- Bootstrap confidence intervals for benchmark metrics if practical.
- Report effect sizes in addition to p-values; the study may be underpowered for formal significance with only three seeds.
- Predefine a practical-importance rule before viewing final results, for example a minimum consistent relative perplexity improvement or percentage-point downstream gain.

6.4 Core result tables

Table	Required contents
Dense baselines	Model scale, total/targeted parameters, perplexity, downstream scores, CPU memory and latency.
Compression controls	Pruning-only and quantization-only outcomes by model and budget.
Sequential vs joint	Absolute metrics, retention, joint gain and uncertainty.
Efficiency	Checkpoint size, effective bits, non-zero count, prefill, decode and throughput.
Ablations	Reverse order, calibration size and optional layer-targeting changes.
External validation	Qwen dense-normalised results only; no raw cross-tokenizer scale inference.

6.5 Core figures

Figure	Purpose
Figure 1	Study design: Pythia scale sweep and optional Qwen validation.
Figure 2	Dense quality and CPU cost versus model scale.
Figure 3	Quality retention of sequential and joint methods by scale.
Figure 4	Joint gain versus log targeted parameter count.
Figure 5	Moderate versus aggressive budget comparison.
Figure 6	Accuracy/perplexity versus checkpoint size Pareto plot.
Figure 7	Quality versus CPU latency or throughput Pareto plot.
Figure 8	Optional Qwen external validation compared through normalised retention.

6.6 Interpretation framework

Observed pattern	Defensible interpretation
Joint gain rises with scale	Supports a scale-dependent benefit; discuss redundancy and error interaction as plausible mechanisms, not proven causes.
Joint gain is flat	Suggests joint optimisation benefit is budget- or method-dependent rather than scale-dependent in the tested range.
Joint gain appears only at aggressive budget	Supports an interaction between scale and compression severity.
Sequential matches or beats joint	A valid negative result; the extra complexity may not be justified for these models/budgets.
Qwen disagrees with Pythia	The Pythia trend may be family-specific; architecture/training differences require broader validation.
Quality improves but latency does not	The scientific compression effect is real, but the CPU runtime does not exploit sparsity or packing efficiently.

6.7 Claims the paper may make

- Within the tested Pythia range, joint gain increased/decreased/remained stable with scale.
- The relationship differed between moderate and aggressive budgets.
- A similar or different direction appeared on Qwen2.5-0.5B.
- The tested CPU backend did or did not convert theoretical compression into deployment speedup.
- The chosen joint method was or was not worth its added compression cost under the tested conditions.

6.8 Claims the paper must not make

- A universal model-size threshold for all SLMs.
- A causal proof that redundancy alone explains the observed trend.
- Generalisations to instruction-tuned, multimodal or mixture-of-experts models.
- Sparse CPU acceleration without actual runtime measurements.
- Cross-family conclusions based on absolute perplexity values from different tokenizers.
- State-of-the-art compression unless all relevant methods and deployment settings were reproduced fairly.

7. Repository, Logging and Reproducibility

7.1 Recommended repository structure

Repository layout

```
joint-compression-scale-study/
configs/
models/
compression/
evaluation/
data_indices/
src/
models.py
capture_activations.py
pruning.py
quantization.py
sequential.py
joint.py
reconstruct.py
evaluate_ppl.py
benchmark_cpu.py
scripts/
results/raw/
results/processed/
plots/
checkpoints/
environment/
README.md
```

7.2 Per-run metadata schema

Category	Fields
Identity	run_id, date, git commit, operator
Model	repository ID, revision hash, tokenizer revision, size label
Hardware	CPU, RAM, GPU, VRAM, OS, power mode
Software	Python, PyTorch, Transformers, evaluation harness, backend versions
Data	dataset, split, sample indices, token count, sequence length
Compression	method, sparsity, bit-width, group size, targeted modules, steps, seed
Quality	perplexity, task scores, retention, logits/reconstruction metrics
Efficiency	file size, peak RAM, prefill, decode, throughput, compression time
Status	success, warning, failure, divergence reason

7.3 Raw artefacts to preserve

- [] All YAML/JSON configuration files.
- [] Calibration sample indices and tokenised input hashes.
- [] Dense and compressed checkpoint manifests.
- [] Per-layer reconstruction loss before and after compression.
- [] Pruning masks and quantization scales/codebooks.
- [] Evaluation-harness JSON outputs.
- [] CPU benchmark raw repetitions, not only summary statistics.
- [] Environment export and exact package versions.
- [] Failure logs and excluded-run reasons.

7.4 Reproducibility checklist

- ☐ A fresh environment can load every final checkpoint.
- ☐ One command reproduces each main table or figure from raw results.
- ☐ The README states the expected disk, RAM and VRAM requirements.
- ☐ Random seeds are set for Python, NumPy and PyTorch.
- ☐ Determinism settings and known non-deterministic operations are documented.
- ☐ Model and dataset licences are recorded.
- ☐ No private API keys or machine-specific absolute paths are committed.

8. Two-to-Three-Week Execution Schedule

8.1 Recommended 21-day schedule

Time	Focus	Deliverable
Days 1-2	Protocol freeze	Finalise RQs, model revisions, data, target layers, metrics, environment and run naming.
Days 2-3	Hardware pilot	Profile 160M/410M/1B; test activation capture and one-layer compression; decide 1.4B.
Days 3-5	Build 160M pipeline	Dense, pruning-only, quantization-only, sequential and joint end to end.
Days 5-6	Budget screening	Run candidate budgets on 160M; confirm promising choices on 410M; freeze two budgets.
Days 7-9	410M full pass	Complete full matrix and first CPU benchmarks.
Days 10-13	1B full pass	Complete mandatory 1B runs; use overnight jobs where safe.
Days 13-15	Confirmatory seeds	Repeat central sequential/joint comparisons across all core scales.
Day 16	Core analysis gate	Generate main tables and scale-effect plots; confirm the paper is complete without optional work.
Days 17-18	Optional extension	Run 1.4B or Qwen validation, not both unless the core is fully secure.
Days 18-20	Write Results and Discussion	Interpret findings, threats to validity and deployment implications.
Day 21	Finalisation	Abstract, conclusion, references, reproducibility audit, figure/table QA and submission package.

8.2 Accelerated 14-day schedule

Day	Action
1	Freeze protocol and environment.
2	Hardware pilot and 1.4B decision.
3-4	Complete 160M implementation and smoke tests.
5	Screen and freeze two budgets.
6-7	Complete 410M experiments.
8-10	Complete 1B experiments.
11	Confirmatory repeats for the central budget only.
12	Analysis, plots and tables.
13	Write Results, Discussion and Limitations.
14	Abstract, Conclusion, references and reproducibility QA.

Schedule protection rule

Qwen and 1.4B are not allowed to consume the time reserved for confirmatory Pythia seeds, analysis, or paper writing. A complete three-scale paper is stronger than an unfinished five-model paper.

8.3 Team split for two researchers

Owner	Primary responsibility
Researcher A	Model/data pipeline, pruning implementation, dense/pruning baselines, CPU benchmark script.
Researcher B	Quantization implementation, joint optimiser, evaluation harness, plotting and result aggregation.
Both	Protocol decisions, sequential baseline, confirmatory runs, interpretation, writing and final review.

9. Complete Paper Headings and Subheadings

Front matter

- Title
- Authors and affiliations
- Abstract
- Keywords

Abstract structure

- Problem: sequential and joint compression may interact differently across model scales.
- Method: controlled Pythia scale sweep with fixed pruning/quantization budgets and optional Qwen validation.
- Results: one sentence with the main scale trend and strongest numerical result.
- Implication: when the added complexity of joint compression is or is not justified.

1. Introduction

Subheading	Content
1.1 Motivation	Deployment cost of decoder-only language models and complementary roles of pruning and quantization.
1.2 Sequential versus joint compression	Why order and interaction may matter.
1.3 Research gap	Single-model studies do not reveal scale-dependent benefit.
1.4 Research questions and hypotheses	State the primary RQ, secondary RQs and hypotheses.
1.5 Contributions	List the controlled scale study, evaluation protocol and deployment analysis.

2. Background and Related Work

Subheading	Content
2.1 Decoder-only transformer language models	Causal attention, transformer blocks and weight-heavy linear layers.
2.2 Neural network pruning	Unstructured, blockwise, structured, magnitude and second-order approaches.
2.3 Weight quantization	PTQ, QAT, per-channel/groupwise schemes and low-bit reconstruction.
2.4 Sequential compression	Order effects and accumulated error.
2.5 Joint pruning and quantization	Co-optimisation methods such as joint search or reconstruction [4,5].
2.6 Model scale, redundancy and compression sensitivity	Why scale may alter damage and recovery capacity.
2.7 Research gap and positioning	What prior work does not answer and what this study tests.

3. Methodology

Subheading	Content
3.1 Study design	Independent variable, treatments, budgets and controlled variables.
3.2 Model selection	Pythia sizes, optional 1.4B, Qwen external validation.
3.3 Layer targeting and parameter accounting	Transformer blocks, exclusions, total versus targeted parameters.

Subheading	Content
3.4 Calibration data	Corpus, tokenisation, sample selection and held-out subset.
3.5 Pruning method	Saliency, mask granularity, target sparsity and reconstruction.
3.6 Quantization method	Bit-width, symmetry, group size, scales and packing.
3.7 Sequential compression	P->Q and reverse-order ablation.
3.8 Joint compression	Common objective and alternating optimisation.
3.9 Compute matching and fairness	Equal data, steps, seeds and module coverage.
3.10 Evaluation metrics	Perplexity, downstream tasks, retention and joint gain.
3.11 CPU benchmarking protocol	Prefill, decode, throughput, memory and repetitions.
3.12 Statistical analysis	Seed policy, uncertainty and descriptive scale trend.

4. Experimental Setup

Subheading	Content
4.1 Hardware	Laptop CPU, RAM, GPU, VRAM and power mode.
4.2 Software	OS, Python, PyTorch, Transformers, evaluation harness and runtime.
4.3 Hyperparameters	Calibration size, sequence length, local steps, optimiser and seeds.
4.4 Compression budgets	Screening procedure and final frozen moderate/aggressive budgets.
4.5 Reproducibility controls	Pinned revisions, configs and deterministic settings.

5. Results

Subheading	Content
5.1 Dense baselines across scale	Quality and CPU cost before compression.
5.2 Isolated pruning and quantization effects	Controls by model and budget.
5.3 Sequential versus joint compression	Direct results at each scale.
5.4 Scale dependence of joint gain	Central figure and effect-size analysis.
5.5 Interaction with compression severity	Moderate versus aggressive budgets.
5.6 CPU deployment outcomes	Size, RAM, prefill, decode and throughput.
5.7 Ablations	Reverse order, calibration size and optional layer targeting.
5.8 External validation on Qwen2.5-0.5B	Normalised result only; optional section.

6. Discussion

Subheading	Content
------------	---------

Subheading	Content
6.1 Main findings	Answer each research question directly.
6.2 Why joint benefit may change with scale	Discuss redundancy, error interaction and optimisation flexibility as hypotheses.
6.3 When joint complexity is justified	Practical decision guidance by model size and budget.
6.4 Quality versus real deployment benefit	Explain backend and sparse-kernel limitations.
6.5 Cross-family evidence	Interpret Qwen agreement or disagreement cautiously.
6.6 Comparison with prior work	Relate results to joint-compression and pruning literature.
6.7 Negative or null results	Explain what a flat or reversed trend means.

7. Limitations and Threats to Validity

Subheading	Content
7.1 Internal validity	Hyperparameter tuning, calibration representativeness, implementation errors.
7.2 Construct validity	Whether reconstruction loss and selected tasks capture useful model quality.
7.3 External validity	One primary family, base models and limited size range.
7.4 Statistical conclusion validity	Few scale points and limited seeds.
7.5 Deployment validity	Laptop CPU and runtime-specific sparse/quantized kernel behaviour.
7.6 Reproducibility limitations	Hardware variance and library-version sensitivity.

8. Conclusion and Future Work

- Restate the main scale finding without overstating causality.
- State which compression strategy is preferable in the tested range.
- Summarise whether CPU deployment benefits matched theoretical compression.
- Recommend broader families, structured sparsity, activation quantization and larger models as future work.

Back matter

- Acknowledgements
- Data and code availability statement
- Conflict of interest statement, if required
- References
- Appendix A: Full hyperparameters
- Appendix B: Complete result tables
- Appendix C: Additional plots and failure cases
- Appendix D: Hardware and benchmark details

10. Risks, Decision Gates and Final Checklists

10.1 Risk register

Risk	Likelihood	Impact	Mitigation
1B/1.4B exceeds VRAM	High	High	Use layerwise processing, smaller activation batches, mixed precision, CPU offload; drop 1.4B before weakening the core design.
Joint method unstable at W4/high sparsity	Medium	High	Use screening, gradient clipping/local step limits, and select a stable aggressive budget.
Sequential baseline is unfairly weak	Medium	High	Compute-match recovery and add reverse-order ablation at the central budget.
CPU sparsity gives no speedup	High	Medium	Report theoretical and measured benefits separately; treat null latency result as valid.
Full task evaluation too slow	Medium	Medium	Use three required tasks, caching and fixed evaluation limits for exploration; run full sets only on final checkpoints.
Scope creep delays writing	High	High	Freeze core at three Pythia models and two budgets; optional work begins only after the analysis gate.
Perplexity comparisons misused across tokenizers	Medium	High	Use within-model normalisation for Qwen and state the limitation explicitly.
Different embedding fractions confound scale	Medium	High	Report targeted non-embedding counts and whole-model effective compression separately.

10.2 First 48-hour checklist

- [] Create the repository and environment lock file.
- [] Record exact HP Omen CPU, RAM, GPU and VRAM.
- [] Download and pin Pythia-160M, 410M and 1B revisions.
- [] Select one Pythia data variant consistently.
- [] Build dense perplexity evaluation and verify against known/expected ranges.
- [] Capture one layer of calibration activations.
- [] Implement and test weight-only quantization on one linear layer.
- [] Implement and test pruning on the same layer.
- [] Implement sequential and joint reconstruction on that layer.
- [] Profile peak memory and projected runtime for 1B.
- [] Decide whether 1.4B remains in the plan.
- [] Freeze the experiment protocol in a version-controlled document.

10.3 Before launching full runs

- [] All dense baselines reproduce across repeated evaluations.
- [] Checkpoint reload tests pass.
- [] Sparsity and bit-width are verified programmatically.
- [] Sequential and joint methods use identical data and compute budgets.
- [] The screening grid has selected exactly two final budgets.
- [] Run IDs and output directories cannot overwrite prior results.
- [] A failed run can resume from a layer or checkpoint.
- [] Disk space is sufficient for dense, compressed and temporary artefacts.

10.4 Before writing the Results section

- [] Every mandatory model/budget condition has a valid result or documented failure.
- [] Confirmatory seeds are complete for the central comparisons.
- [] Raw CPU repetitions are saved.
- [] No optional model is mixed into the Pythia scale regression.
- [] Joint gain is computed using the strongest available sequential baseline.
- [] Figures are generated from scripts, not edited manually.
- [] The analysis includes both quality and real deployment outcomes.

10.5 Final paper checklist

- ☐ The title names decoder-only language models and does not imply all SLMs.
- ☐ The abstract reports the model sizes, budgets and central numerical finding.
- ☐ The Introduction states the gap and contributions clearly.
- ☐ Methodology contains enough detail to reproduce masks and quantization.
- ☐ Results answer each research question directly.
- ☐ Discussion separates evidence from speculative mechanisms.
- ☐ Limitations explicitly address three/four scale points and one primary family.
- ☐ No threshold claim exceeds what the data support.
- ☐ References are complete and consistent.
- ☐ Code/data availability and hardware details are included.
- ☐ All tables and figures are cited in the text.
- ☐ The final repository reproduces the main plots from raw logs.

References and Core Sources

- [1] Biderman, S. et al. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. arXiv:2304.01373.
- [2] EleutherAI. Pythia model cards for the released scaling suite, including Pythia-160M, 410M, 1B and 1.4B. Hugging Face model repository documentation.
- [3] Qwen Team (2024). Qwen2.5 Technical Report. arXiv:2412.15115; and Qwen/Qwen2.5-0.5B model card.
- [4] Qu, X. et al. (2025). Automatic Joint Structured Pruning and Quantization for Efficient Neural Network Training and Compression (GETA). arXiv:2502.16638.
- [5] Zandonati, B. et al. (2023). Towards Optimal Compression: Joint Pruning and Quantization. arXiv:2302.07612.
- [6] Kurtic, E. et al. (2022). The Optimal BERT Surgeon: Scalable and Accurate Second-Order Pruning for Large Language Models. arXiv:2203.07259.
- [7] EleutherAI. Language Model Evaluation Harness documentation and task definitions. GitHub repository: EleutherAI/lm-evaluation-harness.

Reference expansion required during writing

The final paper should add the specific pruning, quantization, order-effect and joint-compression papers used to justify the implemented methods. This execution plan lists only the central sources required to support the study design.

Appendix A. Protocol Freeze Template

Protocol field	Frozen value
Working title	
Primary RQ	
Pythia variants and revisions	
Qwen revision, if used	
Target modules	
Calibration dataset and indices	
Sequence length and token count	
Pruning saliency/granularity	
Final moderate budget	
Final aggressive budget	
Quantization scheme/group size	
Sequential order(s)	
Joint update algorithm	
Local optimisation steps	
Seed list	
Primary evaluation corpus	
Downstream tasks	
CPU runtime and thread count	
Practical-importance threshold	
1.4B go/no-go decision	
Qwen go/no-go decision	

Appendix B. Run Completion Tracker

Model	Condition	Done	Notes
Pythia-160M	Dense	☐	
Pythia-160M	Pruning only	☐	
Pythia-160M	Quantization only	☐	
Pythia-160M	Sequential P->Q	☐	
Pythia-160M	Joint	☐	
Pythia-160M	Reverse Q->P	☐	Optional/central budget
Pythia-410M	Dense	☐	
Pythia-410M	Pruning only	☐	
Pythia-410M	Quantization only	☐	
Pythia-410M	Sequential P->Q	☐	
Pythia-410M	Joint	☐	
Pythia-410M	Reverse Q->P	☐	Optional/central budget
Pythia-1B	Dense	☐	
Pythia-1B	Pruning only	☐	
Pythia-1B	Quantization only	☐	
Pythia-1B	Sequential P->Q	☐	
Pythia-1B	Joint	☐	
Pythia-1B	Reverse Q->P	☐	Optional/central budget
Pythia-1.4B (optional)	Dense	☐	
Pythia-1.4B (optional)	Pruning only	☐	
Pythia-1.4B (optional)	Quantization only	☐	
Pythia-1.4B (optional)	Sequential P->Q	☐	
Pythia-1.4B (optional)	Joint	☐	
Pythia-1.4B (optional)	Reverse Q->P	☐	Optional/central budget
Qwen2.5-0.5B (optional)	Dense	☐	
Qwen2.5-0.5B (optional)	Pruning only	☐	
Qwen2.5-0.5B (optional)	Quantization only	☐	
Qwen2.5-0.5B (optional)	Sequential P->Q	☐	
Qwen2.5-0.5B (optional)	Joint	☐	
Qwen2.5-0.5B (optional)	Reverse Q->P	☐	Optional/central budget

